SECURE WEB APPLICATION USING FLASK

Cyber Security Internship Project Report

Submitted by	Mercy Angelin Mary
Programme	B.E. Cyber Security
Date	September 2026

1. Introduction

Web applications support many online services and must protect user credentials and sessions. This internship project demonstrates a secure Flask prototype with registration, login, protected dashboard access, and logout.

2. Objectives

• Develop a functional Flask web application.
• Implement registration and authentication.
• Store passwords as hashes rather than plain text.
• use parameterized SQL queries and validate user input.
• Publish clear documentation and evidence on GitHub.

3. Tools and Technologies

Python, Flask, SQLite, Werkzeug security utilities, HTML, CSS, GitHub, and a modern web browser.

4. Application Flow

User registration → validated input → hashed password storage → login verification → authenticated session → protected dashboard → secure logout.

5. Security Controls Implemented

Password hashing: Werkzeug generates and verifies password hashes.
Parameterized queries: Values are passed separately to SQLite statements to reduce SQL injection risk.
Session control: The dashboard checks for an authenticated session.
Input validation: Name, email format, and minimum password length are verified.
Logout: Session data is cleared when the user signs out.

6. Demonstration Evidence

The following screenshots were generated from the completed local Flask demonstration. The same source files should be uploaded to the GitHub repository so the screenshots and repository code remain consistent.

7. Testing Summary

Valid registration creates a user record with a password hash. Correct credentials open the dashboard. Incorrect credentials are rejected. Direct dashboard access without a session redirects to the sign-in page. Logout clears the session.

8. Learning Outcomes

The project provided practical experience with Flask routing, HTML forms, SQLite database access, password hashing, session management, validation, GitHub documentation, and security-focused reporting.

9. Limitations and Future Enhancements

This is an educational prototype. Before production use, configure a strong secret key through environment variables, enable HTTPS, add CSRF protection, rate limiting, secure cookie settings, password reset controls, logging, and automated tests.

10. Conclusion

The project demonstrates a working foundation for secure web authentication using Flask. It connects secure coding concepts with a practical application and creates clear evidence suitable for internship evaluation.

Author

Mercy Angelin Mary
B.E. Cyber Security